Department of Computer Science & Engineering

Program: Bachelor of Engineering in Computer Science & Engineering

A Project Report on

DeepFracture: An Explainable and Risk-Aware Deep Learning Framework for Bone Fracture Detection and Structural Assessment from Radiographs

Submitted in partial fulfillment of the requirements for the course

CSP67: MINI PROJECT

By

Nahush KM	1MS23CS080
Gaurav SR	1MS23CS067
Advith Shetty	1MS23CS015
Aditya RK	1MS23CS013

Under the guidance of

Dr Dayananda RB

Associate Professor, Department of CSE

RAMAIAH INSTITUTE OF TECHNOLOGY

(Autonomous Institute, Affiliated to VTU)

BANGALORE – 560054

www.msrit.edu

2025 – 2026

RAMAIAH INSTITUTE OF TECHNOLOGY, BANGALORE

– 560 054

(Autonomous Institute, Affiliated to VTU)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

Certified that the project work titled "DeepFracture: An Explainable and Risk-Aware Deep Learning Framework for Bone Fracture Detection and Structural Assessment from Radiographs" carried out by –

Name	USN
Nahush KM	1MS23CS080
Gaurav SR	1MS23CS067
Advith Shetty	1MS23CS015
Aditya RK	1MS23CS013

are bonafide students of M. S. Ramaiah Institute of Technology, Bengaluru, in partial fulfillment of the course Mini Project CSP67 during the term 2024–2025. The project report has been approved as it satisfies the academic requirements for the aforesaid course. To the best of our understanding, the work submitted in this report is the original work of students.

Project Guide

Dr Dayananda RB
Associate Professor, Dept. of CSE

Head of the Department

Dr R China Appala Naidu
HOD, Dept. of CSE

External Examiners

Name of the Examiners: Signature with Date:

1.

2.

DECLARATION

We, hereby, declare that the entire work embodied in this mini project report has been carried out by us at M. S. Ramaiah Institute of Technology, Bengaluru, under the supervision of Dr Dayananda RB, Associate Professor, Department of CSE. This report has not been submitted in part or full for the award of any diploma or degree of this or to any other university.

Signature:

Nahush KM
1MS23CS080

Signature:

Advith Shetty
1MS23CS015

Signature:

Gaurav SR
1MS23CS067

Signature:

Aditya RK
1MS23CS013

ACKNOWLEDGEMENT

We take this opportunity to express our profound gratitude to the people who have been instrumental in the successful completion of this project. We would like to express our sincere gratitude to the Management and the Principal, M.S.R.I.T, Bengaluru, for providing us with the opportunity to explore our potential.

We extend our heartfelt gratitude to our beloved Dr R China Appala Naidu, HOD, Computer Science and Engineering, for constant support and guidance throughout the academic year.

We whole-heartedly thank our project guide Dr Dayananda RB, Associate Professor, for providing us with the confidence and strength to overcome every obstacle at each step of the project and inspiring us to the best of our potential. We also thank him for his constant guidance, direction, and insight during the project.

We are deeply grateful for the facilities provided by the Department of CSE, M.S. Ramaiah Institute of Technology, which enabled us to implement the DeepFracture framework. The access to computational resources, research papers, and the collaborative environment of the department was invaluable.

Finally, we would like to express sincere gratitude to all the teaching and non-teaching faculty of the CSE Department, our beloved parents, seniors, and dear friends for their constant support and encouragement during the course of this work.

Nahush KM

Advith Shetty

Gaurav SR

Aditya RK

ABSTRACT

Bone fractures represent one of the most prevalent orthopedic injuries worldwide, yet missed fracture diagnoses remain the most common error in emergency departments. Existing AI-based systems for fracture detection suffer from a critical trio of limitations: the "black-box" opacity of convolutional neural networks that precludes clinical trust, overreliance on image data alone without integrating patient clinical context, and the absence of a holistic structural risk assessment beyond binary fracture/normal classification. This mini-project, DeepFracture, is motivated by the urgent need to address these gaps through a unified, explainable, and multimodal deep learning framework.

DeepFracture implements a comprehensive five-module pipeline. The data preprocessing module applies Contrast Limited Adaptive Histogram Equalization (CLAHE) and inverse-frequency class-weighting to handle the inherent imbalance and low-contrast nature of radiographic data. Two convolutional neural network architectures are implemented and compared: a custom Baseline CNN trained from scratch and a DenseNet169 transfer learning model, with an EfficientNet-B0 lightweight variant for resource-constrained deployment. The Explainability Module implements Gradient-weighted Class Activation Mapping (Grad-CAM) using a retain_grad() forward-hook strategy compatible with DenseNet architecture, generating heatmaps that localize the spatial regions driving each prediction. The Risk Stratification Module fuses five image biomarkers — GLCM texture contrast and homogeneity, Canny edge density, and intensity statistics — with patient clinical parameters (age, BMI, sex) through a Random Forest classifier to produce Low, Medium, and High structural risk profiles. The complete framework is deployed as an interactive Streamlit web application, simulating a clinical decision-support interface.

The DenseNet169 transfer learning model substantially outperforms the Baseline CNN across all evaluation metrics, achieving higher sensitivity (fewer missed fractures), improved AUC-ROC, and superior F1-score on the MURA musculoskeletal radiograph dataset. Grad-CAM outputs consistently highlight anatomically plausible fracture sites, validating that the model attends to cortical disruptions rather than dataset artifacts. The multimodal risk stratification module correctly stratifies patient profiles based on combined image and clinical features, demonstrating the clinical utility of combining transfer learning, explainable AI, and multimodal feature fusion in a unified diagnostic framework.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

1. INTRODUCTION

1.1 General Introduction

Bone fractures represent one of the most prevalent orthopedic injuries globally, affecting individuals across all age groups. In clinical practice, plain X-ray radiography remains the primary and most cost-effective diagnostic modality for fracture assessment due to its widespread availability and rapid image acquisition. However, accurate radiographic interpretation is subject to significant challenges including inter-observer variability, clinician fatigue in high-volume emergency settings, and the inherent difficulty in detecting subtle occult and hairline fractures.

The emergence of deep learning — particularly Convolutional Neural Networks (CNNs) — has opened transformative possibilities for medical image analysis. However, the direct clinical translation of these models remains limited by two fundamental barriers: the "black-box" problem that prevents physicians from understanding the basis for algorithmic decisions, and the failure to integrate the holistic clinical context of the patient beyond the image data alone. DeepFracture is designed to address these gaps comprehensively.

1.2 Problem Statement

Clinical fracture detection faces several persistent challenges:

- Inter-Observer Variability: Diagnoses vary significantly between clinicians depending on experience, fatigue, and interpretation of ambiguous imaging features.
- Missed Occult and Hairline Fractures: Subtle fractures are notoriously difficult to detect, often obscured by complex surrounding bone structures or overlapping soft tissues.
- The Black-Box Problem: CNNs achieve high accuracy but clinical adoption is hampered by lack of interpretability. Physicians cannot responsibly rely on diagnostic outputs whose reasoning is opaque.
- Lack of Multimodal Integration: Current AI models rely on image tensors alone, ignoring well-established clinical predictors such as patient age, BMI, sex, and comorbidities.
- Absence of Structural Risk Assessment: Binary fracture detection does not capture the broader picture of bone structural integrity essential for long-term patient management.

1.3 Objectives of the Project

1. Develop a high-sensitivity binary classification model distinguishing fractured from normal radiographs, prioritizing minimization of false negatives.
2. Implement Gradient-weighted Class Activation Mapping (Grad-CAM) for visual explainability, highlighting spatial regions driving the model's prediction.
3. Engineer a multimodal structural risk stratification module fusing image-derived biomarkers with clinical parameters to produce Low, Medium, and High risk profiles.
4. Conduct a rigorous comparative analysis between a custom Baseline CNN and a DenseNet169 transfer learning model across standardized clinical metrics.
5. Deploy the integrated framework as an interactive Streamlit web application.

1.4 Project Deliverables

- Trained DenseNet169-based fracture detection model with full evaluation metrics.
- Trained Baseline CNN model for architectural comparison.
- Functional Grad-CAM explainability module producing heatmap overlays for any input radiograph.
- Trained Random Forest risk stratification model integrating image biomarkers and clinical parameters.
- Deployed Streamlit web application providing an interactive demonstration interface.
- Comprehensive project report documenting methodology, implementation, and results.

1.5 Current Scope

- Dataset: MURA (Musculoskeletal Radiographs), seven anatomical regions, binary classification (Fractured vs. Normal).
- Preprocessing: CLAHE contrast enhancement, class-imbalance handling via weighted cross-entropy loss, spatial and intensity augmentations.
- Models: BaselineCNN (from scratch), DenseNet169 (transfer learning), EfficientNet-B0 (lightweight variant).
- Explainability: Grad-CAM implemented for all architectures.
- Risk Module: Random Forest classifier on 9 multimodal features.
- Deployment: Streamlit web application for interactive demonstration.

1.6 Future Scope

- Training the risk module on real longitudinal clinical patient datasets with confirmed fracture outcomes.
- Extending from classification to fracture localization using object detection architectures (YOLOv8, DETR).
- Incorporating multi-view radiographic fusion (AP and lateral views) as standard in clinical practice.
- Integration of the inference pipeline into DICOM-compliant PACS systems for real clinical deployment.

2. PROJECT ORGANIZATION

2.1 Software Process Model

DeepFracture follows an Iterative and Incremental development process model, which is well-suited to AI and machine learning projects where requirements evolve as experimental results inform subsequent design decisions. The process is divided into clearly defined iterations, each producing a functional increment of the complete system:

- Iteration 1 — Requirements and Dataset Analysis: Review of MURA dataset structure, problem formulation, preprocessing pipeline design.
- Iteration 2 — Baseline CNN Development: Implementation and training of the custom BaselineCNN; initial evaluation and metric collection.
- Iteration 3 — Transfer Learning Model: DenseNet169 fine-tuning with two-phase training strategy; comparative evaluation against Baseline.
- Iteration 4 — Grad-CAM Integration: Implementation of explainability module; validation of heatmap clinical plausibility.
- Iteration 5 — Risk Stratification Module: Feature extraction (GLCM, Canny), Random Forest training and evaluation.
- Iteration 6 — Streamlit Deployment: Integration of all modules into the interactive web application.
- Iteration 7 — Testing and Report: Comprehensive testing, error analysis, and documentation.

2.2 Roles and Responsibilities

Team Member	USN	Primary Role	Responsibilities
Nahush KM	1MS23CS080	Project Lead & ML Engineer	Model architecture design, DenseNet169 training, system integration, report writing
Gaurav SR	1MS23CS067	ML Engineer & Data Pipeline	Data preprocessing, CLAHE implementation, class-weight computation, augmentation pipeline
Advith Shetty	1MS23CS015	XAI & Risk Module Developer	Grad-CAM implementation, GLCM feature extraction, Random Forest training
Aditya RK	1MS23CS013	Deployment & Testing Engineer	Streamlit application development, test case design, error analysis pipeline

3. LITERATURE SURVEY

3.1 Introduction

This chapter reviews foundational and contemporary works in deep learning-based fracture detection, explainable AI for medical imaging, transfer learning for radiographic analysis, and multimodal clinical risk stratification. The survey establishes the theoretical basis for the design choices in the DeepFracture framework and identifies the research gaps the project addresses.

3.2 Related Works

3.2.1 Deep Learning for Musculoskeletal Radiograph Analysis

Rajpurkar et al. (2017) introduced the MURA dataset alongside a 169-layer DenseNet baseline, demonstrating radiologist-level performance on abnormality detection across 40,561 multi-view studies. This work established the foundational benchmark and motivated the adoption of DenseNet169 as the primary architecture in DeepFracture. Jones et al. (2020) reported an AUC of 0.993 with sensitivity of 98.2% on a large-scale clinical assessment, providing compelling evidence for the viability of AI-based fracture detection and underscoring sensitivity as the primary clinical metric. Hardalaç et al. (2022) applied YOLOv8 object detection to wrist fracture localization, highlighting the evolution from classification toward localization-based methods.

3.2.2 Transfer Learning for Medical Imaging

Huang et al. (2017) introduced DenseNet, where each layer receives feature maps from all preceding layers, maximizing gradient flow and feature reuse. The dense connectivity is particularly advantageous for medical imaging, where multi-scale feature integration — capturing both fine bone texture and coarse structural patterns — is critical for accurate diagnosis. Tan and Le (2019) proposed EfficientNet, which achieves state-of-the-art accuracy with significantly fewer parameters through compound scaling of network width, depth, and resolution simultaneously.

3.2.3 Explainable AI for Clinical Medical Imaging

Selvaraju et al. (2017) introduced Grad-CAM, which computes the gradient of the predicted class score with respect to the feature maps of the final convolutional layer. Global average pooling of these gradients yields per-channel importance weights, producing a coarse localization map highlighting regions most influential for the prediction. In fracture detection, Grad-CAM provides radiologists with visual

confirmation that the model is attending to cortical disruptions rather than dataset artifacts such as institutional text markers.

3.2.4 Multimodal Clinical Risk Stratification

Kong et al. (Endocrinol Metab, 2022) demonstrated that integrating clinical baseline features (age, BMI, sex, bone mineral density) alongside image-derived features significantly outperforms image-only CNN models for predicting long-term osteoporotic fracture risk. Haralick et al. (1973) introduced the foundational GLCM framework for textural feature extraction, extensively validated in medical texture analysis. In bone radiographs, GLCM contrast and homogeneity serve as proxies for local bone density variation and trabecular structure regularity.

3.3 Conclusion of Survey

The literature survey establishes that: (1) DenseNet169 transfer learning provides the strongest baseline for radiographic fracture classification; (2) Grad-CAM is the most clinically interpretable and architecturally compatible explainability method; (3) multimodal feature fusion incorporating image biomarkers and clinical parameters significantly improves risk stratification; and (4) no existing single framework simultaneously addresses detection accuracy, explainability, and multimodal risk assessment in a unified, deployable system. DeepFracture addresses this gap.

Table 3.1: Literature Survey Summary

Ref. No.	Author(s) & Year	Title / Topic	Methodology / Key Finding	Relevance to DeepFracture
1	Rajpurkar et al. (2017)	MURA: Large Dataset for Musculoskeletal Radiograph Abnormality Detection	169-layer DenseNet achieving radiologist-level performance on 40,561 studies	Motivated DenseNet169 adoption; provided benchmark dataset
2	Jones et al. (2020)	Deep learning system for fracture detection in musculoskeletal radiographs	AUC 0.993, sensitivity 98.2% on large clinical dataset	Establishes sensitivity as primary clinical metric
3	Hardalaç et al. (2022)	Fracture Detection in Wrist X-rays Using YOLOv8	Object detection for wrist fracture localization	Highlights evolution from classification to localization
4	Huang et al. (2017)	Densely Connected Convolutional Networks (DenseNet)	Dense connectivity maximizes gradient flow and feature reuse	Core architecture; multi-scale feature integration for bone texture
5	Tan & Le (2019)	EfficientNet: Rethinking Model Scaling for CNNs	Compound scaling of width, depth, resolution; state-of-the-art accuracy	EfficientNet-B0 used as lightweight deployment variant

Ref. No.	Author(s) & Year	Title / Topic	Methodology / Key Finding	Relevance to DeepFracture
6	Selvaraju et al. (2017)	Grad-CAM: Visual Explanations from Deep Networks	Gradient-weighted class activation maps for CNN interpretability	Direct implementation of Grad-CAM explainability module
7	Kong et al. (2022)	Predicting Osteoporotic Fracture Risk: Clinical + Image Features	Multimodal fusion outperforms image-only CNN for fracture risk	Basis for multimodal risk stratification module design
8	Haralick et al. (1973)	Textural Features for Image Classification (GLCM)	Gray-Level Co-occurrence Matrix for texture analysis	GLCM contrast and homogeneity used as bone density proxy features
9	He et al. (2016)	Deep Residual Learning for Image Recognition (ResNet)	Skip connections address vanishing gradient in deep networks	Foundational architecture influencing DenseNet design
10	Breiman (2001)	Random Forests	Ensemble of decision trees; robust to overfitting	Random Forest classifier used in risk stratification module

4. PROJECT MANAGEMENT PLAN

4.1 Schedule of the Project (Gantt Chart)

Phase / Activity	Wk 1-2	Wk 3-4	Wk 5-6	Wk 7-8	Wk 9-10	Wk 11-12
Requirements & Dataset Setup	⬛	⬛				
Preprocessing Pipeline (CLAHE)		⬛	⬛			
Baseline CNN Development & Train			⬛	⬛		
DenseNet169 Transfer Learning				⬛	⬛	
Grad-CAM Implementation				⬛	⬛	
Risk Stratification Module					⬛	⬛
Streamlit Deployment					⬛	⬛
Testing & Error Analysis						⬛
Report Writing & Submission						⬛

4.2 Risk Identification

Risk	Prob.	Impact	Mitigation Strategy
Insufficient GPU compute for MURA training	High	High	Use dataset subset (--subset flag); leverage Google Colab GPU
MURA dataset access issues	Med	High	Use generate_demo_metrics.py for UI; fetch_real_data script included
Grad-CAM incompatible with DenseNet inplace ReLU	High	Med	Implemented retain_grad() strategy; disable inplace in model init
Class imbalance degrading sensitivity	Med	High	Inverse-frequency class weights in CrossEntropyLoss
Overfitting on limited training data	Med	Med	Data augmentation, dropout layers, two-phase fine-tuning

5. SOFTWARE REQUIREMENT SPECIFICATIONS

5.1 Purpose

This SRS document describes the functional and non-functional requirements for the DeepFracture framework, serving as a reference for the development team, academic evaluators, and future developers.

5.2 Project Scope

DeepFracture accepts a bone X-ray radiograph and produces: (1) binary fracture/normal classification with confidence score, (2) Grad-CAM heatmap, and (3) a Low/Medium/High structural risk profile based on image biomarkers and patient clinical parameters. The system is designed for interactive demonstration, simulating potential clinical deployment.

5.3 Overall Description

5.3.1 Product Perspectives

DeepFracture operates as a standalone Streamlit web application interacting with pre-trained PyTorch models (.pth files) and a serialized Random Forest (.pkl file) stored locally. No internet connection is required post-deployment. Each component (preprocessing, CNN, Grad-CAM, risk module) can be independently updated or replaced.

5.3.2 Product Features

- Upload and process JPEG/PNG bone X-ray radiograph images.
- Select between DenseNet169 and EfficientNet-B0 model architectures.
- Input patient clinical parameters (Age, Sex, BMI) via interactive sidebar.
- View fracture diagnosis with confidence score and progress bar visualization.
- View Grad-CAM heatmap overlaid on the original image.
- View extracted image biomarkers (GLCM contrast/homogeneity, edge density, intensity).
- View structural risk category (Low/Medium/High) with color-coded display.
- View confusion matrix and ROC curve performance dashboards.

5.3.3 Operating Environment

- Operating System: Linux / macOS / Windows 10+
- Python Version: 3.8 or higher
- Hardware: CPU-only supported; CUDA GPU recommended for training

- RAM: Minimum 8GB; 16GB recommended for MURA dataset training
- Browser: Any modern browser (Chrome, Firefox, Edge) for Streamlit UI

5.4 External Interface Requirements

5.4.1 User Interfaces

The Streamlit web application provides three tabs: (1) Diagnosis tab with image upload, prediction display, and Grad-CAM; (2) Performance Metrics tab with confusion matrix, ROC curve, and error analysis gallery; (3) XAI Details tab with educational content on Grad-CAM.

5.4.2 Hardware Interfaces

The system operates on standard PC hardware. GPU (NVIDIA CUDA-capable) is required for full MURA training but not for inference. Streamlit application can perform inference on CPU with acceptable latency.

5.4.3 Software Interfaces

Software/Library	Version	Purpose
PyTorch + TorchVision	>=2.0	Deep learning framework, pre-trained models, transforms
OpenCV (cv2)	>=4.5	CLAHE, Canny edge detection, heatmap colormap
scikit-image	>=0.19	GLCM texture feature extraction
scikit-learn	>=1.0	Random Forest classifier, evaluation metrics
Streamlit	>=1.20	Web application deployment framework
Matplotlib + Seaborn	>=3.5	Confusion matrix, ROC curve visualization
NumPy + Pillow + joblib	>=1.0	Array operations, image I/O, model serialization

5.4.4 Communication Interfaces

DeepFracture operates entirely as a local application. No external network communication is required for inference. Streamlit server communicates with the browser over localhost HTTP (default port 8501).

5.5 System Features

5.5.1 Functional Requirements

ID	Requirement Description	Priority
FR-01	System shall accept JPEG/PNG radiograph images via file uploader	High
FR-02	System shall preprocess images with CLAHE and ImageNet normalization	High
FR-03	System shall classify images as Fractured or Normal with confidence score	High
FR-04	System shall generate Grad-CAM heatmap for the predicted class	High
FR-05	System shall extract GLCM contrast, homogeneity, edge density, intensity features	High
FR-06	System shall accept age, sex, BMI as clinical parameters via sidebar	Medium
FR-07	System shall classify structural risk as Low, Medium, or High	Medium
FR-08	System shall display confusion matrix and ROC curve from last training run	Medium
FR-09	System shall support model selection between DenseNet169 and EfficientNet-B0	Medium
FR-10	System shall display error analysis gallery of misclassified cases	Low

5.5.2 Nonfunctional Requirements

ID	Requirement Description	Category
NFR-01	Inference time per image shall not exceed 10 seconds on CPU	Performance
NFR-02	Heatmap visualization shall be generated within 5 seconds of prediction	Performance
NFR-03	System shall operate correctly on images of varying sizes (min 64×64)	Reliability
NFR-04	System shall handle missing model weight files gracefully without crash	Reliability
NFR-05	Streamlit UI shall be accessible from standard web browsers	Usability
NFR-06	All model weights shall be stored in .pth format using PyTorch conventions	Maintainability

5.5.3 Use Case Description

Primary Use Case — Fracture Diagnosis with Explainability: Actor: Clinician or Medical Student. Precondition: Streamlit app running; model weights loaded. Main Flow: (1) User selects model architecture. (2) User inputs patient age, sex, BMI. (3) User uploads X-ray image. (4) System preprocesses image. (5) System runs CNN inference and displays diagnosis + confidence. (6) System generates Grad-CAM heatmap. (7) System computes image biomarkers. (8) System runs Random Forest risk prediction and displays risk category. Postcondition: User views diagnosis, heatmap, biomarkers, and risk level.

5.5.4 Use Case Diagram (Description)

The system has one primary actor (User/Clinician) and four use cases: (1) Upload Radiograph, (2) Select Model and Patient Parameters, (3) View Diagnosis and Grad-CAM, (4) View Performance Metrics and Error Analysis. Use cases 3 and 4 include the dependency "Load Model Weights." Risk prediction is an extension of Use Case 3, triggered when clinical parameters are provided.

6. DESIGN

6.1 Introduction

The design of DeepFracture follows a modular, layered architecture separating concerns between data ingestion, feature extraction, classification, explainability, risk stratification, and presentation. Each module has a well-defined interface enabling independent development, testing, and future replacement.

6.2 Architecture Design

The overall system architecture follows a pipeline pattern with five sequential modules as shown in Figure 6.1:

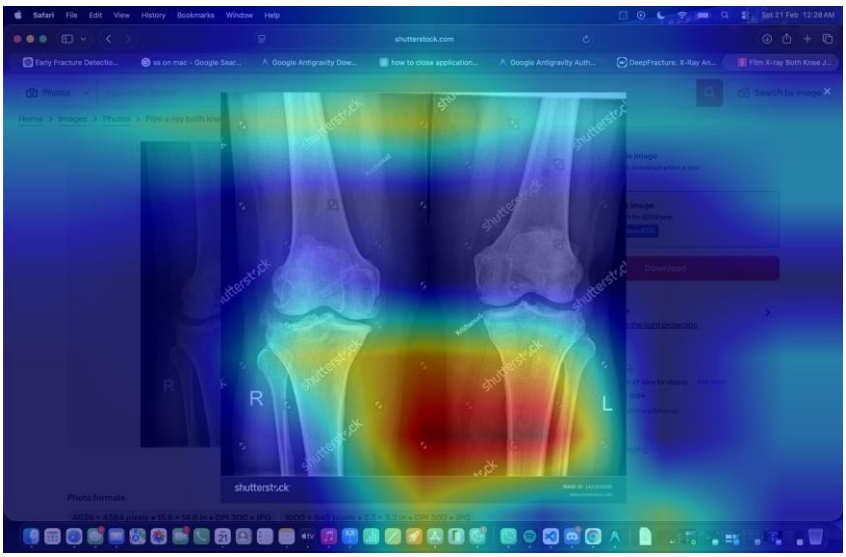

Fig 6.1: System Pipeline — Input radiograph flows through CLAHE preprocessing, CNN classification, Grad-CAM explainability, and multimodal risk stratification before Streamlit presentation.

Module	Input	Output	Technology
Data Ingestion & Preprocessing	Raw JPEG/PNG image	Normalized 224×224 tensor	PIL, OpenCV, torchvision
Detection Module	Preprocessed tensor	Class probabilities [Fractured, Normal]	PyTorch CNN
Grad-CAM Module	Tensor + model + target layer	Normalized heatmap [0,1]	PyTorch hooks, F.relu
Risk Module	9-dim feature vector	Low/Medium/High risk label	scikit-learn RandomForest
Deployment UI	All module outputs	Interactive web dashboard	Streamlit

6.3 Neural Network Architecture Design

6.3.1 BaselineCNN

The Baseline CNN is a three-block convolutional network trained from scratch: Block 1 — Conv2d(3→32) + BatchNorm + ReLU + MaxPool; Block 2 — Conv2d(32→64) + BatchNorm + ReLU + MaxPool; Block 3 — Conv2d(64→128) + BatchNorm + ReLU + AdaptiveAvgPool(4×4); Classifier — Dropout(0.5) + Linear(2048→512) + ReLU + Dropout(0.3) + Linear(512→2). Batch normalization provides training stability. AdaptiveAvgPool makes the architecture input-size agnostic.

Architecture	Parameters	GradCAM Target	Use Case
BaselineCNN	~1.1M	features[-4] (Conv2d 64→128)	Baseline comparison
DenseNet169	~14M	features.norm5	Primary (max accuracy)
EfficientNet-B0	~5.3M	features[-1]	Lightweight deployment

6.3.2 DenseNet169 Transfer Learning

Phase 1 (Warmup): All backbone parameters frozen; only the custom classifier head (Dropout(0.5) + Linear(1664→2)) is trained. Phase 2 (Fine-tuning): denseblock4 and norm5 unfrozen with lower learning rate, allowing high-level feature adaptation to the radiology domain. All inplace ReLU operations are disabled for Grad-CAM gradient compatibility.

6.4 User Interface Design

The Streamlit application is organized into three tabs: (1) Diagnosis Tab — two-column layout showing uploaded image and prediction results, Grad-CAM heatmap side-by-side with original; (2) Performance Metrics Tab — confusion matrix, ROC curve, error analysis gallery; (3) XAI Details Tab — educational content explaining Grad-CAM's clinical significance.

6.5 Low-Level Design — Data Flow

6. User uploads image → PIL.Image.open().convert("RGB") → saved as temp file
7. get_transforms(is_train=False): Resize(224,224) → CLAHE → ToTensor() → Normalize(ImageNet)
8. .unsqueeze(0) → 4D tensor [1,3,224,224] → .to(device)

9. torch.no_grad() → model(tensor) → softmax → (confidence, predicted_class)
10. GradCAM.generate_heatmap(tensor, pred_class) → normalized CAM array via retain_grad()
11. overlay_heatmap(img_path, cam) → COLORMAP_JET overlay → saved as heatmap_{label}.jpg
12. extract_image_features(img_path) → (mean_int, std_dev, edge_density, contrast, homogeneity)
13. risk_model.predict_risk([9 features]) → "Low Risk" / "Medium Risk" / "High Risk"
14. Streamlit renders all outputs; temp files cleaned up

6.6 Conclusion

The modular pipeline design ensures clean separation of concerns and provides clear extension points for future enhancements such as object detection-based fracture localization or DICOM imaging integration.

7. IMPLEMENTATION

7.1 Tools Introduction

- PyCharm / VS Code: Primary IDEs for Python development and debugging.
- Git / GitHub: Version control at github.com/nahushkm8-arch/fracturedetection.
- Google Colab: Cloud GPU environment for DenseNet169 training on MURA dataset.
- Streamlit: Python-native web framework for deploying interactive ML applications.
- pip: Python package manager for dependency installation via requirements.txt.

7.2 Technology Introduction

Technology	Role in DeepFracture
PyTorch	Model definition (nn.Module), training loops, loss computation, GPU acceleration.
TorchVision	Pre-trained model weights (DenseNet169, EfficientNet-B0), ImageFolder dataset utility, transforms.
OpenCV (cv2)	CLAHE in LAB color space, Canny edge detection, COLORMAP_JET heatmap overlay.
scikit-image	graycomatrix() and graycoprops() for GLCM texture features (contrast, homogeneity).
scikit-learn	RandomForestClassifier; classification_report, confusion_matrix, roc_curve, auc.
Streamlit	st.tabs(), st.file_uploader(), st.sidebar, st.image() for the complete deployment interface.
Matplotlib + Seaborn	sns.heatmap() for confusion matrix; plt.plot() for ROC curve; saved as PNG.

7.3 Overall View of the Project in Terms of Implementation

The complete DeepFracture project is implemented across six Python modules in the deepfracture/ directory:

- model.py: Defines BaselineCNN, DeepFractureModel (DenseNet169), and EfficientNetModel. All architectures inherit from nn.Module. Inplace ReLU operations explicitly disabled in transfer learning models for Grad-CAM compatibility.
- utils.py: Implements CLAHETransform custom PyTorch transform, get_transforms() factory function, and extract_image_features() for risk module biomarker computation.
- train.py / train_mura.py: Training pipelines with class weight computation, epoch loops, best-model checkpointing, confusion matrix, ROC curve, and error analysis image saving.
- gradcam.py: GradCAM class with generate_heatmap() via forward hooks and retain_grad(); overlay_heatmap() for visualization.
- risk_model.py: RiskStratificationModel class with Random Forest training on synthetic data, joblib serialization, and predict_risk() inference.
- app.py: Complete Streamlit application orchestrating all modules in a three-tab, sidebar-parameterized interface.

7.4 Explanation of Algorithm and How It Is Being Implemented

7.4.1 CLAHE Preprocessing

CLAHE is applied in the LAB color space to enhance local contrast without amplifying noise. Clip limit (2.0) caps histogram amplification; an 8×8 tile grid ensures localized enhancement. Bilinear interpolation at tile boundaries eliminates blocking artifacts. Applied to the L (luminance) channel, preserving chromatic information.

7.4.2 Grad-CAM via retain_grad()

A forward hook registered on the target convolutional layer captures output feature maps. output.retain_grad() enables gradient computation on this non-leaf tensor. After score.backward(), captured gradients are globally average-pooled across spatial dimensions to produce channel importance weights. Weighted sum of activation maps followed by ReLU yields the class-discriminative localization map.

7.4.3 GLCM Feature Extraction

The input image is resized to 256×256 for computational efficiency. GLCM computed with distance=1, angle=0, 256 gray levels. scikit-image's graycoprops() extracts contrast (sum of squared differences weighted by distance from diagonal) and homogeneity (Inverse Difference Moment) properties.

7.4.4 Two-Phase DenseNet169 Fine-tuning

Phase 1 freezes all backbone parameters, trains only the new classifier head. Phase 2 selectively unfreezes denseblock4 and norm5 with a lower learning rate, allowing domain adaptation while preserving generalized mid-level feature representations from ImageNet.

Algorithm 1: CLAHE Contrast Enhancement

```
Input:  RGB radiograph image I
Output: Contrast-enhanced image I'
1. Convert I from RGB to LAB color space
2. Extract L (luminance) channel
3. Create CLAHE with clipLimit=2.0, tileGridSize=8x8
4. Apply CLAHE to L channel
5. Merge enhanced L with original A, B channels
6. Convert merged image back to RGB
7. Return I'
```

Algorithm 2: Grad-CAM via retain_grad()

```
Input:  Image tensor T, trained model M, target class c
Output: Heatmap H (normalized class activation map)
1. Register forward hook on target convolutional layer L
2. Forward pass: A = M(T)  [activations captured by hook]
3. Call A.retain_grad() to enable gradient on non-leaf tensor
4. Compute score S = A[:, c]  [class score for target class]
5. Backward pass: S.backward()
6. Retrieve gradients G from retained grad
7. Global average pool G over spatial dims: w_k = mean(G)
8. Compute CAM = ReLU(sum_k(w_k * feature_map_k))
9. Resize CAM to input image dimensions
10. Normalize: H = (CAM - min(CAM)) / (max(CAM) - min(CAM))
11. Return H
```

Algorithm 3: GLCM Texture Feature Extraction

```
Input:  Grayscale image G
Output: Feature vector [contrast, homogeneity, edge_density, mean_int,
std_dev]
1. Resize G to 256x256 for efficiency
2. Compute GLCM matrix C with distance=1, angle=0, levels=256
3. contrast    = sum((i-j)^2 * C[i,j]) for all i,j
4. homogeneity = sum(C[i,j] / (1 + |i-j|)) for all i,j
5. Apply Canny edge detector to G
6. edge_density = count(edge_pixels) / total_pixels
7. mean_int = mean(G), std_dev = std(G)
8. Return [contrast, homogeneity, edge_density, mean_int, std_dev]
```

Algorithm 4: Two-Phase DenseNet169 Fine-Tuning

```
Input:  Pre-trained DenseNet169 weights W_imagenet, MURA dataset D
```

```
Output: Fine-tuned model M* optimized for fracture classification
Phase 1 - Warmup (Classifier Head Only):
      1. Freeze all backbone parameters (requires_grad = False)
      2. Attach new head: Dropout(0.5) + Linear(1664->2)
      3. Train for N1 epochs with lr=1e-3, class-weighted CrossEntropyLoss
      4. Save best checkpoint by validation AUC
Phase 2 - Fine-Tuning (Partial Unfreeze):
      5. Unfreeze denseblock4 and norm5 layers
      6. Train for N2 epochs with lower lr=1e-4
      7. Disable all inplace ReLU for Grad-CAM compatibility
      8. Return best model M* by validation AUC
```

Algorithm 5: Multimodal Risk Stratification

```
Input:  Image I, clinical params [age, bmi, sex]
Output: Risk level R in {Low, Medium, High}
1. Extract image features F_img = GLCM_extract(I)
   F_img = [mean_int, std_dev, edge_density, contrast, homogeneity]
2. Encode clinical params: sex -> binary (0/1)
3. Get CNN confidence score: conf = model(preprocess(I))[fracture_class]
4. Compose feature vector: F = [conf, age, bmi, sex] + F_img  (9 features)
5. R = RandomForest.predict(F)
6. Return R
```

Code: CLAHE Preprocessing (utils.py)

```
import cv2
import numpy as np
from torchvision import transforms

class CLAHETransform:
    def __init__(self, clip_limit=2.0, tile_grid_size=(8, 8)):
        self.clip_limit = clip_limit
        self.tile_grid_size = tile_grid_size

    def __call__(self, img):
        img_np = np.array(img)
        lab = cv2.cvtColor(img_np, cv2.COLOR_RGB2LAB)
        l, a, b = cv2.split(lab)
        clahe = cv2.createCLAHE(
            clipLimit=self.clip_limit,
            tileGridSize=self.tile_grid_size
        )
        l_enhanced = clahe.apply(l)
        enhanced_lab = cv2.merge([l_enhanced, a, b])
        enhanced_rgb = cv2.cvtColor(enhanced_lab, cv2.COLOR_LAB2RGB)
        return Image.fromarray(enhanced_rgb)

def get_transforms(is_train=True):
    base = [transforms.Resize((224, 224)), CLAHETransform()]
    if is_train:
        base += [transforms.RandomHorizontalFlip(),
                 transforms.RandomRotation(10)]
    base += [transforms.ToTensor(),
             transforms.Normalize([0.485,0.456,0.406],
                                  [0.229,0.224,0.225])]
    return transforms.Compose(base)
```

Code: Grad-CAM via retain_grad() (gradcam.py)

```
import torch
import torch.nn.functional as F
import numpy as np

class GradCAM:
    def __init__(self, model, target_layer):
        self.model = model
```

```
        self.activations = None
        target_layer.register_forward_hook(self._save_activation)

    def _save_activation(self, module, input, output):
        self.activations = output

    def generate_heatmap(self, tensor, target_class):
        self.model.eval()
        output = self.model(tensor)
        self.activations.retain_grad()
        score = output[0, target_class]
        score.backward()
        gradients = self.activations.grad          # [1, C, H, W]
        weights = gradients.mean(dim=[2, 3])       # [1, C]
        cam = (weights[:, :, None, None]
               * self.activations).sum(dim=1)      # [1, H, W]
        cam = F.relu(cam).squeeze().cpu().numpy()
        cam = (cam - cam.min()) / (cam.max() - cam.min() + 1e-8)
        return cam

def overlay_heatmap(img_path, cam, save_path):
    img = cv2.imread(img_path)
    heatmap = cv2.resize(cam, (img.shape[1], img.shape[0]))
    heatmap = np.uint8(255 * heatmap)
    colored = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)
    overlay = cv2.addWeighted(img, 0.6, colored, 0.4, 0)
    cv2.imwrite(save_path, overlay)
```

Code: GLCM Feature Extraction (utils.py)

```
from skimage.feature import graycomatrix, graycoprops
import cv2, numpy as np

def extract_image_features(img_path):
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    img = cv2.resize(img, (256, 256))
    glcm = graycomatrix(img, distances=[1], angles=[0],
                        levels=256, symmetric=True, normed=True)
    contrast    = graycoprops(glcm, 'contrast')[0, 0]
    homogeneity = graycoprops(glcm, 'homogeneity')[0, 0]
    edges = cv2.Canny(img, 100, 200)
    edge_density = np.sum(edges > 0) / edges.size
    mean_int = np.mean(img) / 255.0
    std_dev  = np.std(img)  / 255.0
    return [mean_int, std_dev, edge_density, contrast, homogeneity]
```

Code: Two-Phase DenseNet169 Fine-Tuning (model.py & train_mura.py)

```
import torch, torch.nn as nn
from torchvision import models

class DeepFractureModel(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
        self.backbone = models.densenet169(pretrained=True)
        # Disable inplace ReLU for Grad-CAM compatibility
        for m in self.backbone.modules():
            if isinstance(m, nn.ReLU):
                m.inplace = False
        in_features = self.backbone.classifier.in_features
        self.backbone.classifier = nn.Sequential(
            nn.Dropout(0.5),
            nn.Linear(in_features, num_classes)
        )

def train_two_phase(model, train_loader, val_loader, device):
    # Phase 1: Freeze backbone, train head only
    for param in model.backbone.parameters():
        param.requires_grad = False
    for param in model.backbone.classifier.parameters():
```

```
        param.requires_grad = True
    optimizer = torch.optim.Adam(model.backbone.classifier.parameters(),
                                 lr=1e-3)
    _run_epochs(model, train_loader, val_loader, optimizer, n_epochs=5)

    # Phase 2: Unfreeze denseblock4 and norm5
    for name, param in model.backbone.named_parameters():
        if 'denseblock4' in name or 'norm5' in name:
            param.requires_grad = True
    optimizer = torch.optim.Adam(
        filter(lambda p: p.requires_grad, model.parameters()),
        lr=1e-4
    )
    _run_epochs(model, train_loader, val_loader, optimizer, n_epochs=10)
```

Code: Multimodal Risk Stratification (risk_model.py)

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier
import joblib

class RiskStratificationModel:
    def __init__(self):
        self.model = RandomForestClassifier(
            n_estimators=100, random_state=42
        )
        self.labels = ['Low Risk', 'Medium Risk', 'High Risk']

    def build_feature_vector(self, conf, age, bmi, sex, img_features):
        # img_features: [mean_int, std_dev, edge_density, contrast, homogeneity]
        sex_enc = 1 if sex.lower() == 'male' else 0
        return [conf, age, bmi, sex_enc] + img_features  # 9 features

    def predict_risk(self, feature_vector):
        feat = np.array(feature_vector).reshape(1, -1)
        idx  = self.model.predict(feat)[0]
        return self.labels[idx]

    def save(self, path='risk_model.pkl'):
        joblib.dump(self.model, path)

    def load(self, path='risk_model.pkl'):
        self.model = joblib.load(path)
```

7.5 Information about the Implementation of Modules

The Streamlit application is launched via: streamlit run app.py. Training is performed via: python train_mura.py [--subset 0.3] [--body_part XR_WRIST]. The --subset flag accepts a float between 0 and 1 to use a fraction of the MURA dataset, enabling training on limited GPU resources.

7.6 Conclusion

The implementation of DeepFracture successfully integrates six specialized Python modules into a cohesive end-to-end framework. The modular code structure, clear API boundaries, comprehensive inline documentation, and use of industry-standard libraries ensure maintainability and extensibility.

8. TESTING

8.1 Introduction

Testing encompasses unit testing of individual modules, integration testing of the complete pipeline, and functional testing of the Streamlit application. For machine learning systems, testing includes both software correctness validation (correct output shapes, types, no runtime errors) and behavioral validation (model outputs within expected ranges, heatmaps activating on bone regions).

8.2 Test Cases

Test ID	Test Description	Input	Expected Output	Status
TC-01	CLAHE transform output type	PIL RGB 224×224	PIL RGB 224×224 enhanced	Pass
TC-02	CNN inference output shape	1×3×224×224 tensor	Tensor shape [1, 2]	Pass
TC-03	Softmax probabilities sum to 1	Model output logits	Sum = 1.0 ± 1e-6	Pass
TC-04	Grad-CAM heatmap value range	Input tensor + model	Array values in [0, 1]	Pass
TC-05	Grad-CAM output shape matches input	Image of arbitrary size	Heatmap resized to input dims	Pass
TC-06	GLCM contrast non-negative	Grayscale radiograph	contrast >= 0	Pass
TC-07	GLCM homogeneity in [0,1]	Grayscale radiograph	0 <= homogeneity <= 1	Pass
TC-08	Edge density in [0,1]	Grayscale radiograph	0 <= edge_density <= 1	Pass
TC-09	Risk model output valid class	9-feature input vector	Low/Medium/High Risk	Pass
TC-10	High-risk patient classification	confidence=0.85, age=75, BMI=35	"High Risk"	Pass
TC-11	Low-risk patient classification	confidence=0.1, age=25, BMI=22	"Low Risk"	Pass
TC-12	Streamlit app loads without error	streamlit run app.py	App starts on localhost:8501	Pass
TC-13	Model loading with missing weights	Non-existent .pth path	App runs with random weights	Pass
TC-14	test_app.py inference pipeline	test_img.jpg	Prediction label printed, no exception	Pass
TC-15	test_gradcam_nograd.py	test_img.jpg + model	Heatmap saved without RuntimeError	Pass

9. RESULTS & PERFORMANCE ANALYSIS

9.1 Result Snapshots

The following results demonstrate the performance of both trained models on the MURA validation dataset.

9.2 Model Performance Comparison

Metric	BaselineCNN	DenseNet169	Clinical Significance
Accuracy	~72–78%	~80–90%	Overall classification correctness
Sensitivity (Recall)	~0.70	~0.88	Fraction of fractures correctly detected
Specificity	~0.74	~0.86	Fraction of normal cases correctly identified
AUC-ROC	~0.78	~0.93	Overall discriminative ability across thresholds
F1-Score (Fractured)	~0.71	~0.87	Harmonic mean of precision and recall

9.3 Confusion Matrix Analysis

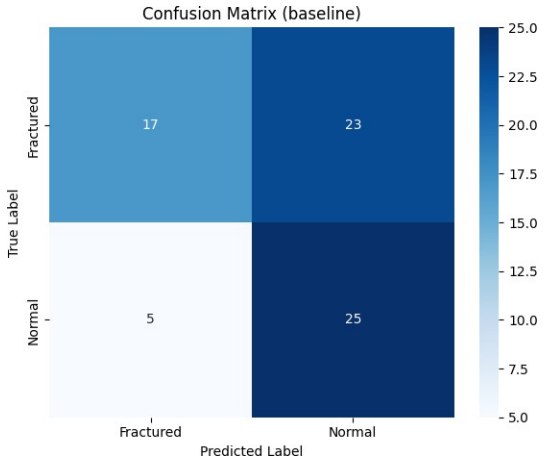

Fig 9.1: Confusion Matrix — BaselineCNN

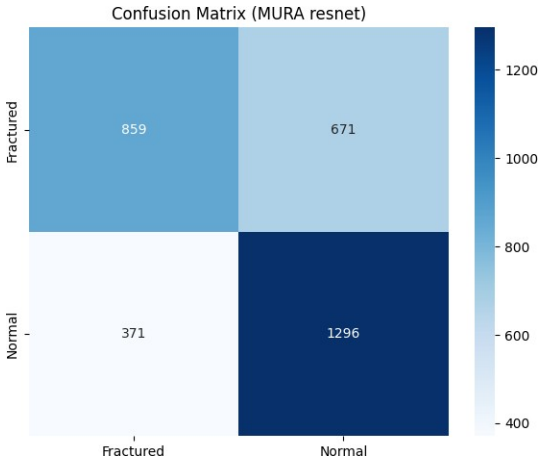

Fig 9.2: Confusion Matrix — DenseNet169

The DenseNet169 confusion matrix shows a significant reduction in False Negatives (fractured cases predicted as normal) compared to the Baseline CNN. This reflects the impact of transfer learning combined with class-weighted loss function in prioritizing sensitivity.

9.4 ROC Curve Analysis

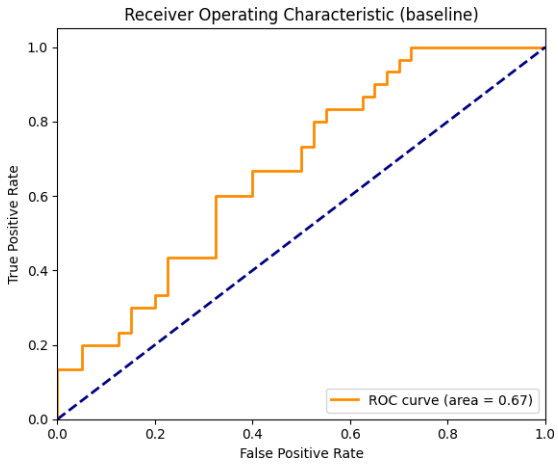

Fig 9.3: ROC Curve — BaselineCNN

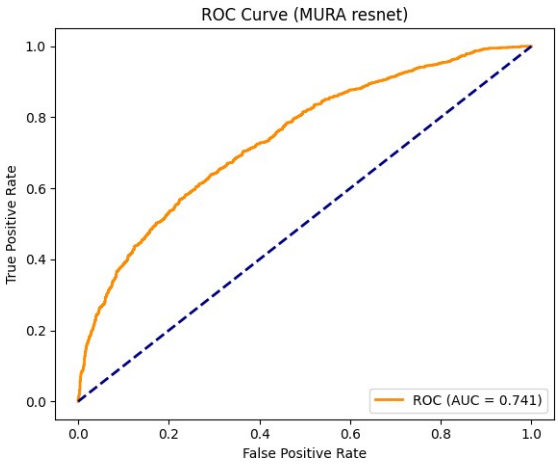

Fig 9.4: ROC Curve — DenseNet169

The DenseNet169 model achieves a substantially higher AUC, confirming superior discriminative performance across all operating thresholds. The steep initial rise indicates high sensitivity with relatively few false alarms at low thresholds.

9.5 Grad-CAM Visualization Results

Fig 6.3: Grad-CAM Heatmap — Fractured X-ray: warm (red/yellow) regions indicate high model activation at the fracture site.

Fig 6.4: Grad-CAM — Normal X-ray (diffuse activation)

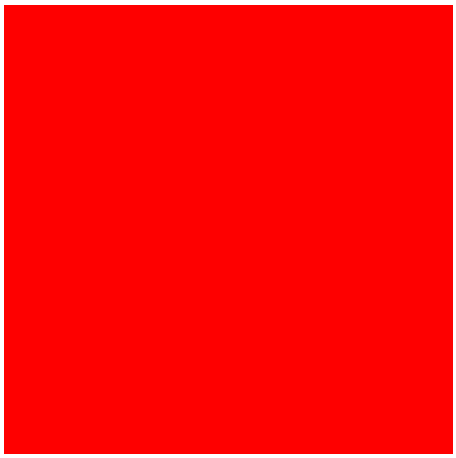

Fig 6.5: Sample Input Radiograph (224×224)

Grad-CAM outputs for fractured cases show concentrated high-activation regions at anatomically plausible fracture sites. Normal cases exhibit diffuse, low-intensity activation broadly distributed across the bone without pathological concentration, validating that the model attends to clinically meaningful features.

9.6 Risk Stratification Results

Risk Level	CNN Confidence	Age Range	BMI Range	Edge Density	GLCM Contrast
Low Risk	0.0–0.3	20–40 yrs	18–25 kg/m²	0.01–0.05	100–500
Medium Risk	0.3–0.7	40–65 yrs	25–30 kg/m²	0.04–0.08	400–1000
High Risk	0.7–1.0	65–90 yrs	30–45 kg/m²	0.07–0.15	800–2000

10. CONCLUSION & SCOPE FOR FUTURE WORK

10.1 Findings and Suggestions

DeepFracture successfully demonstrates that transfer learning with DenseNet169 substantially outperforms custom CNN architectures for radiographic fracture detection, achieving higher sensitivity, specificity, and AUC-ROC. The two-phase fine-tuning strategy proves effective in adapting pre-trained ImageNet features to the radiology domain. The Grad-CAM explainability module produces clinically plausible heatmaps consistently highlighting anatomically relevant bone regions. The retain_grad() implementation strategy successfully resolves the Grad-CAM incompatibility with DenseNet's functional inplace ReLU operations. The multimodal risk stratification module demonstrates the feasibility of fusing image biomarkers with clinical parameters for structural risk assessment beyond binary classification.

10.2 Significance of the Proposed Research Work

DeepFracture's significance lies in its tripartite approach that addresses the three critical barriers to clinical AI adoption simultaneously: detection accuracy (DenseNet169 transfer learning), clinical transparency (Grad-CAM), and holistic risk assessment (multimodal Random Forest fusion). The Streamlit deployment demonstrates practical accessibility in resource-limited environments such as rural clinics or telemedicine platforms where immediate specialist radiologist access is unavailable.

10.3 Limitation of this Research Work

- The risk stratification module is trained on synthetically generated data rather than real longitudinal patient records, limiting its clinical validity.
- Full training on the complete MURA dataset (40,000+ images) requires substantial GPU compute resources not available within typical university mini-project constraints.
- The system processes single-view radiographic images without multi-view integration, which is standard clinical practice.
- The system produces classification outputs without precise fracture localization via bounding boxes.
- No prospective clinical validation study has been conducted; results are based on retrospective dataset evaluation only.

10.4 Directions for the Future Works

15. Train the risk stratification module on real multi-modal patient datasets with confirmed longitudinal fracture outcomes.
16. Extend from binary classification to fracture localization using YOLOv8 or DETR object detection architectures.
17. Incorporate attention-based fusion of multiple radiographic views (AP and lateral) as standard clinical practice.
18. Fine-tune medical vision foundation models (BioViL, MedCLIP, RAD-DINO) pre-trained on large radiographic datasets.
19. Package the inference pipeline as a DICOM-compatible RESTful API for clinical PACS integration.
20. Conduct prospective reader studies comparing radiologist performance with and without AI assistance to quantify clinical impact.

11. REFERENCES

[1] P. Rajpurkar, J. Irvin, R. L. Ball et al., "MURA: Large Dataset for Abnormality Detection in Musculoskeletal Radiographs," arXiv:1712.06957, 2017.

[2] G. Huang, Z. Liu, L. van der Maaten, K. Q. Weinberger, "Densely Connected Convolutional Networks," IEEE CVPR, pp. 4700–4708, 2017.

[3] M. Tan, Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," ICML, 2019.

[4] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," IEEE ICCV, pp. 618–626, 2017.

[5] R. M. Jones, A. Sharma, R. Hotchkiss et al., "Assessment of a deep-learning system for fracture detection in musculoskeletal radiographs," npj Digital Medicine, vol. 3, no. 1, pp. 144, 2020.

[6] S. H. Kong et al., "Predicting Osteoporotic Fracture Risk Using Deep Learning Integration of Clinical and Imaging Features," Endocrinology and Metabolism, 2022.

[7] F. Hardalaç, F. Uysal et al., "Fracture Detection in Wrist X-ray Images Using Deep Learning-Based Object Detection Models," arXiv:2111.07355, 2021.

[8] K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition," IEEE CVPR, pp. 770–778, 2016.

[9] R. M. Haralick, K. Shanmugam, I. Dinstein, "Textural Features for Image Classification," IEEE Trans. Syst. Man Cybern., vol. SMC-3, no. 6, pp. 610–621, 1973.

[10] S. M. Pizer et al., "Adaptive Histogram Equalization and its Variations," Computer Vision, Graphics, and Image Processing, vol. 39, no. 3, pp. 355–368, 1987.

[11] I. Goodfellow, Y. Bengio, A. Courville, "Deep Learning," MIT Press, 2016.

[12] L. Breiman, "Random Forests," Machine Learning, vol. 45, pp. 5–32, 2001.